

Able to Process petabyte-scale data using Hadoop/Spark, Choose appropriate time-series databases for IoT/telemetry, <u>Implement</u> blockchain-based data storage solutions, Compare modern data architectures for different use cases	Unit VI: Big Data & Modern Database Technologies 6.1 Hadoop Ecosystem (HDFS, MapReduce) 6.2 Apache Spark (RDDs, <u>DataFrames</u>, Spark SQL) 6.3 Time-Series Databases and its implementation 6.4 Blockchain Databases (Decentralized Storage)	7 hrs
--	---	---------------------

By: Yuba Raj Devkota

Course: Database Engineering

NCIT

What is Big Data?

Big Data requires distributed storage and processing frameworks

Big Data refers to very large, fast, and complex datasets that cannot be stored, processed, or analyzed efficiently using traditional databases or single computers.

Characteristics of Big Data (5 V's)

1. **Volume** – Huge amount of data (TBs, PBs)
2. **Velocity** – Data generated very fast (real-time/streaming)
3. **Variety** – Different formats (text, images, videos, logs)
4. **Veracity** – Data may be noisy or uncertain
5. **Value** – Useful insights extracted from data

YouTube

- Millions of users upload videos every minute
- Data includes:
 - Videos (unstructured)
 - Comments (text)
 - Likes, views, timestamps (structured)

➡ This data is **too large and fast** for normal databases, so **Big Data technologies** (Hadoop, Spark) are used.

Big Data is data that is too large, fast, and complex to be handled by traditional data processing systems.

Weather Monitoring System

- Sensors record temperature every second
- Continuous data over years
- Requires big data storage and analysis

Hadoop Ecosystem (HDFS, MapReduce)

1. Introduction to Hadoop

Hadoop is an open-source framework designed for distributed storage and processing of large datasets across clusters of computers. It is highly scalable and fault-tolerant.

Key Features:

- **Scalability:** Can scale from a single server to thousands of machines.
- **Fault Tolerance:** Automatically handles hardware failures.
- **Cost-Effective:** Uses commodity hardware.
- **Flexibility:** Can store and process structured, semi-structured, and unstructured data.

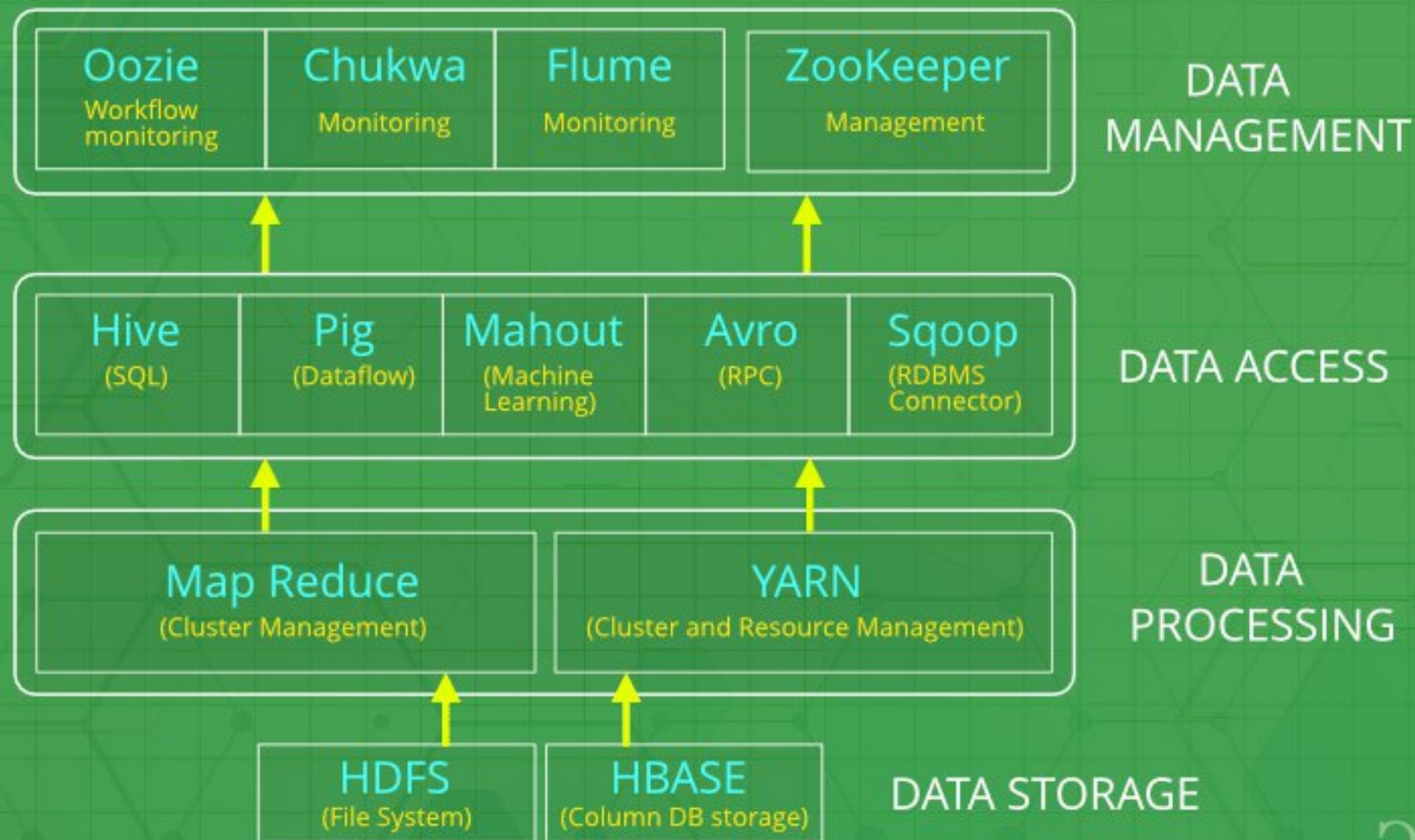
Hadoop Ecosystem is a set of tools that enables distributed storage, processing, management, and analysis of big data using HDFS, MapReduce, YARN, and supporting components.

2. Hadoop Ecosystem Components

The Hadoop ecosystem consists of several components, but the two core components are:

- **HDFS (Hadoop Distributed File System)** – Storage layer
- **MapReduce** – Processing layer

Hadoop Ecosystem



The Hadoop Ecosystem is a collection of tools and frameworks that work together to store, process, manage, and analyze big data in a distributed environment. It is organized into four main layers:

1. Data Storage Layer

- **HDFS (Hadoop Distributed File System)**

Stores large files across multiple machines with fault tolerance.

- **HBase**

A column-oriented NoSQL database that runs on HDFS for real-time read/write access.

 *Purpose:* Reliable and scalable data storage.

2. Data Processing Layer

- **MapReduce**

Processes large datasets using the Map and Reduce programming model.

- **YARN (Yet Another Resource Negotiator)**

Manages cluster resources and schedules jobs.

 *Purpose:* Distributed data processing and resource management.

3. Data Access Layer

- **Hive** – SQL-like querying (HiveQL) for data analysis
- **Pig** – Data flow scripting language (Pig Latin)
- **Mahout** – Machine learning algorithms
- **Avro** – Data serialization and RPC
- **Sqoop** – Transfers data between RDBMS and Hadoop

✦ *Purpose:* Easy data querying, analysis, and integration.

4. Data Management / Coordination Layer

- **Oozie** – Workflow scheduling and job coordination
- **Flume** – Collects streaming/log data into HDFS
- **Chukwa** – Monitoring and data collection
- **ZooKeeper** – Coordination, configuration, and synchronization

✦ *Purpose:* Workflow management, data ingestion, and coordination.

↔ *Overall Flow*

Data is stored in HDFS/HBase → processed by MapReduce using YARN → accessed via Hive/Pig → managed and monitored using Oozie, Flume, ZooKeeper.

Hadoop Ecosystem is a set of tools that enables distributed storage, processing, management, and analysis of big data using HDFS, MapReduce, YARN, and supporting components.

A. HDFS (Hadoop Distributed File System)

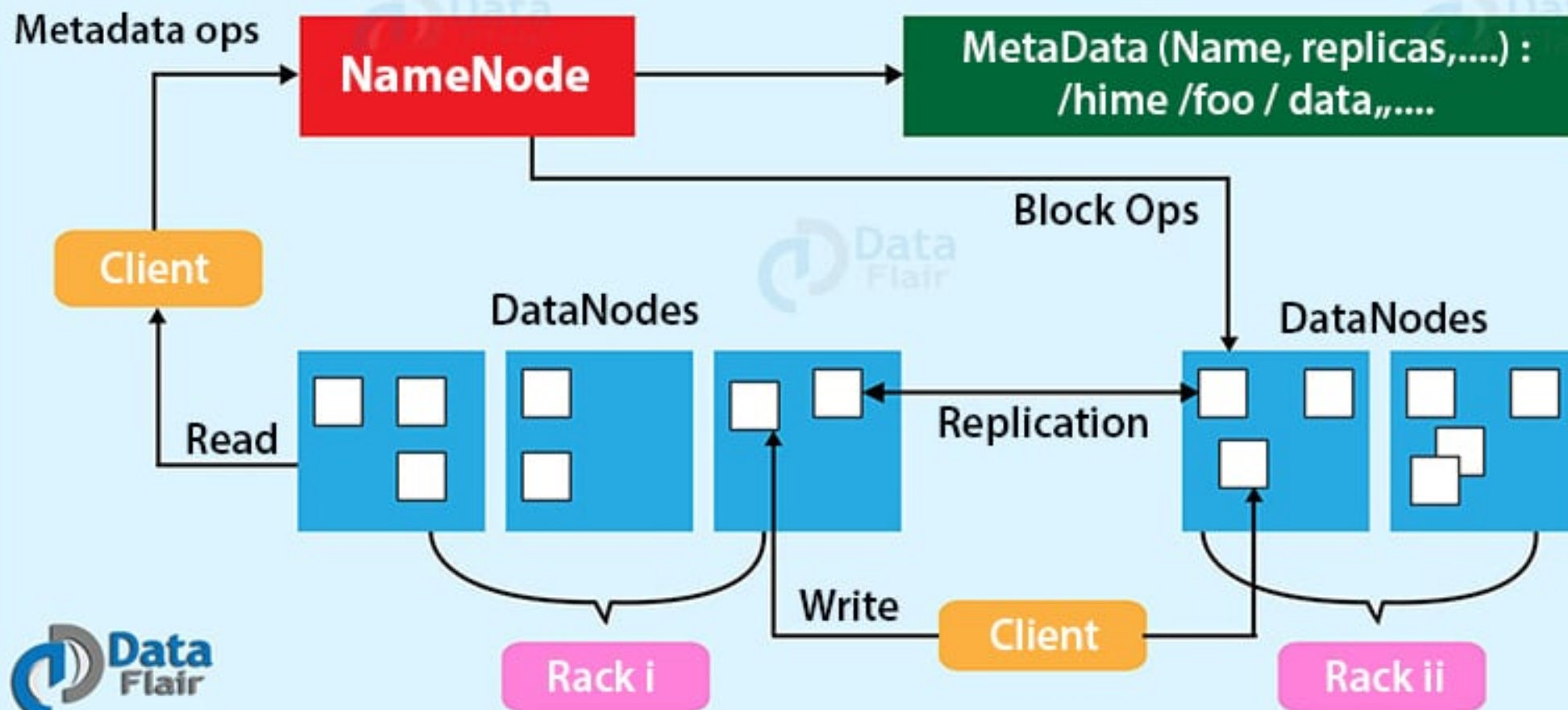
HDFS is the storage system of Hadoop, designed to store large files across multiple machines.

Key Features of HDFS:

- **Distributed Storage:** Splits files into blocks (default 128MB or 256MB) and stores them across a cluster.
- **Replication:** Each block is replicated (default 3 times) for fault tolerance.
- **Master-Slave Architecture:**
 - **NameNode (Master):** Manages metadata (file names, locations, permissions).
 - **DataNode (Slave):** Stores actual data blocks.

The diagram illustrates the core components and workflow of the Hadoop Distributed File System (HDFS), which is designed for storing and managing large-scale data across distributed clusters.

HDFS Architecture



1. Key Components

- **NameNode (Master):**
 - Stores **metadata** (file names, directory structure, block locations, replication factor).
 - Does not store actual data; only tracks where data blocks reside.
 - Handles client requests for file operations (e.g., open, close, rename).
- **DataNodes (Slaves):**
 - Store the **actual data blocks** (e.g., 128MB chunks of files).
 - Report block health and locations to the NameNode periodically.
 - Perform read/write operations as instructed by clients.
- **Client:**
 - Interacts with HDFS to read/write files.
 - Communicates with the NameNode for metadata and DataNodes for block operations.

2. Workflow Explained

A. Read Operation:

1. Client asks the NameNode for block locations of a file (e.g., `/home/foo`).
2. NameNode returns the list of DataNodes holding the blocks.
3. Client reads data directly from the DataNodes (e.g., "Back i" and "Back ii").

B. Write Operation:

1. Client requests the NameNode to create a new file.
2. NameNode allocates DataNodes for block storage (e.g., "DataNodes Back i").
3. Client writes data to the first DataNode, which **replicates** blocks to other nodes (default: 3 copies).

How HDFS Handles Big Data?

1. Distributed Storage:

- Splits files into blocks → stores across multiple machines.
- Parallel read/write operations.

2. Fault Tolerance:

- Replicates each block (default: 3 copies).
- If one DataNode fails, data is read from replicas.

3. Scalability:

- Add more DataNodes to increase storage capacity.

3. Key Concepts

- **Replication:**

- Each block is copied to multiple DataNodes (e.g., 3x) for **fault tolerance**.
- If one DataNode fails, data is still accessible from replicas.

- **Metadata Ops:**

- NameNode manages metadata operations (e.g., listing files, creating directories).

- **Block Ops:**

- DataNodes handle low-level block storage and retrieval.

B. Map Reduce

MapReduce is a programming model for processing large datasets in parallel across a Hadoop cluster.

Phases of MapReduce:

1. Map Phase:

- Processes input data and generates key-value pairs.
- Example: Counting word frequency (input → (word, 1) pairs).

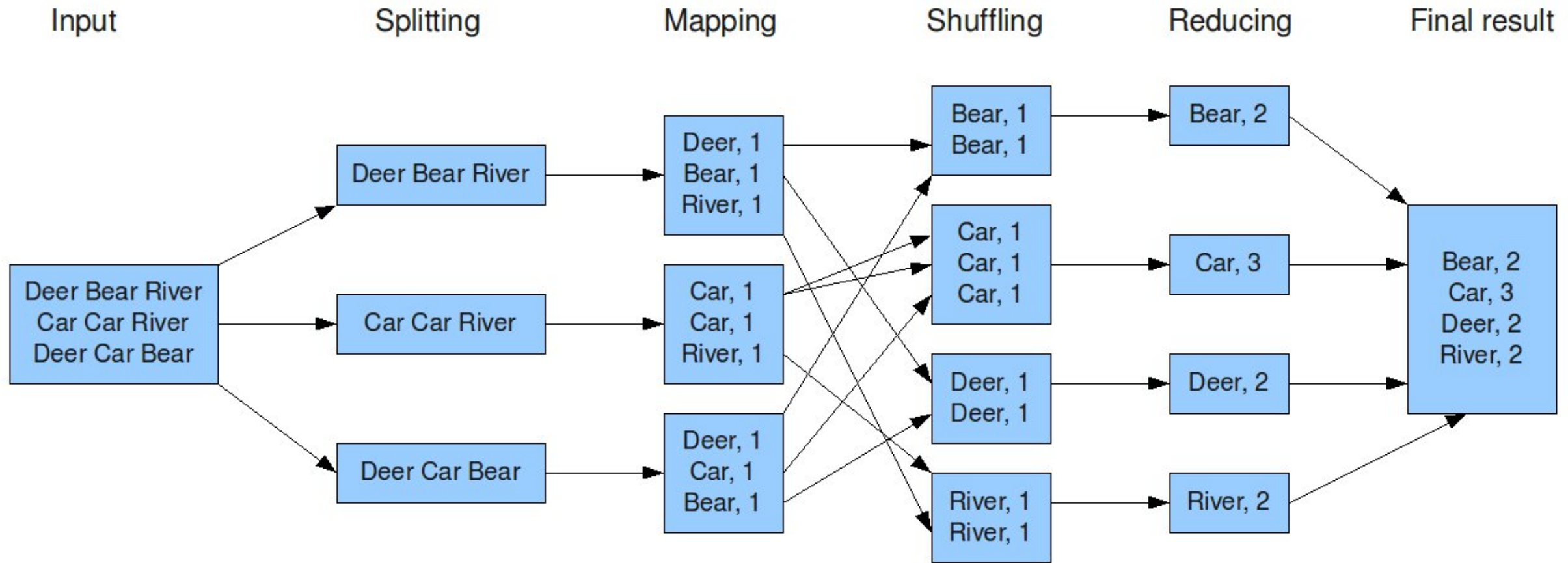
2. Shuffle & Sort Phase:

- Groups intermediate key-value pairs by key.

3. Reduce Phase:

- Aggregates values for each key (e.g., sums counts for each word).

The overall MapReduce word count process



Scenario:

A college wants to count how many times each subject name appears in students' feedback forms collected from different departments.

Input (Feedback Text Files)

arduino

File 1: "DBMS is useful DBMS is important"

File 2: "AI is useful"

File 3: "DBMS and AI are important"

👉 Output of Map phase:

SCSS

(DBMS,1), (DBMS,1), (AI,1), (AI,1), (important,1), ...

1 Map Phase

Each mapper reads text and emits (word, 1)

SCSS

(DBMS, 1)

(is, 1)

(useful, 1)

(DBMS, 1)

(is, 1)

(important, 1)

...

(AI, 1)

...

2 Shuffle & Sort

System groups same keys together automatically:

CSS

DBMS → [1,1]

AI → [1,1]

important → [1,1]

useful → [1,1]

3 Reduce Phase

Reducer adds the values:

nginx

DBMS → 2

AI → 2

important → 2

useful → 2

✓ Final Output

makefile

DBMS: 2

AI: 2

important: 2

useful: 2

💡 Why this is “MapReduce”?

- *Map: Breaks large data into key-value pairs*
- *Reduce: Aggregates results*
- *Works efficiently even if feedback comes from thousands of colleges*

Example: Storing & Processing a Large File in HDFS

Scenario:

A 1GB log file (`server_logs.txt`) needs to be stored in HDFS and processed for error analysis.

Step 1: Store the File in HDFS

Command: Upload the file from local storage to HDFS.

```
bash
```

```
hadoop fs -put /local/path/server_logs.txt /user/hadoop/logs/
```

What Happens?

1. HDFS splits `server_logs.txt` into **blocks** (default: 128MB each → ~8 blocks).
2. Each block is stored across **DataNodes** (e.g., Block1 → DN1, DN2, DN3).
3. **NameNode** records metadata (e.g., file path, block locations, replication factor).

Step 2: Process the File with MapReduce

Goal: Count how many times the word "ERROR" appears in the log.

MapReduce Job (Java/Python-like Pseudocode):

python

```
# Mapper: Emits (key=word, value=1) for each "ERROR"
def mapper(line):
    if "ERROR" in line:
        yield ("ERROR", 1)

# Reducer: Sums counts for each key
def reducer(key, values):
    yield (key, sum(values))
```

Run the Job:

bash

```
hadoop jar hadoop-mapreduce-examples.jar wordcount \
/user/hadoop/logs/server_logs.txt \
/user/hadoop/output/
```

Output:

A file in HDFS (/user/hadoop/output/part-r-00000) with the result:

text

ERROR 1420

Apache Spark (RDDs, DataFrames, Spark SQL)

Apache Spark is an open-source distributed data processing framework used for fast, large-scale data processing. It performs in-memory computation, which makes it much faster than Hadoop MapReduce. Spark supports:

- *Batch processing*
- *Real-time / streaming data*
- *Machine Learning*
- *Graph processing*
- *SQL-like queries*

Spark Core Components

Spark mainly provides three ways to process data:

1. RDDs
2. DataFrames
3. Spark SQL

Key Difference

- 👉 Hadoop MapReduce writes intermediate results to disk,
- 👉 Spark keeps data in memory (RAM) → 10–100x faster

Hadoop vs Apache Spark

Feature	Hadoop	Apache Spark
Storage	HDFS	HDFS, S3, Local, etc.
Processing Model	MapReduce (disk-based)	In-memory processing
Speed	Slower	Much faster
Ease of Use	Complex (Java code)	Easy (Scala, Python, SQL)
Iterative Processing	Poor	Very good
Fault Tolerance	Yes	Yes
Use Case	Large batch jobs	Fast analytics, ML, streaming

An RDD is:

- A **collection of data**
- Distributed across multiple machines
- **Immutable** (cannot be changed)
- **Fault-tolerant**

1. RDD (Resilient Distributed Dataset)

RDD is a low-level data structure in Spark that represents an immutable, distributed collection of objects processed in parallel.

Suppose we have a text file:

kotlin

```
big data is powerful
spark is fast
big data spark
```

RDD Example (Conceptual)

python

```
rdd = sc.textFile("data.txt")
words = rdd.flatMap(lambda line: line.split(" "))
pairs = words.map(lambda w: (w, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

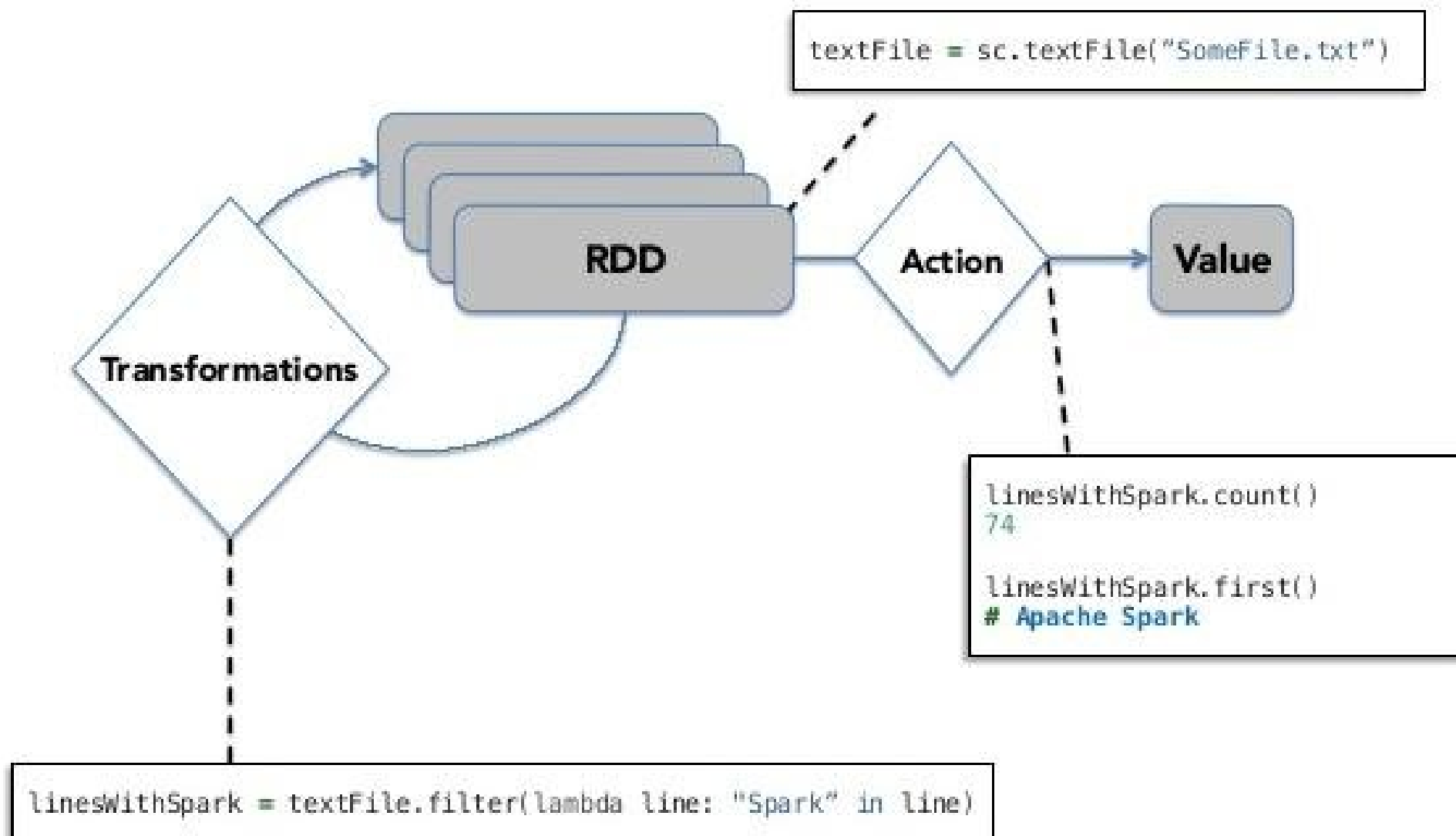
Output:

SCSS

```
(big, 2)
(data, 2)
(spark, 2)
(is, 2)
```

Key Points: More control | More coding | Used internally by Spark | Not beginner-friendly

Working With RDDs



1 Transformations

- Operations like `map()`, `filter()`, `flatMap()`
- **Do not execute immediately**
- They only **define a logical plan**
- Spark builds a **DAG (Directed Acyclic Graph)**

Example

```
scala
```

```
val words = rdd.flatMap(_.split(" "))
```

➡ No computation happens yet

2 Lazy Evaluation

- Spark delays execution
- Improves performance by **optimizing execution plan**
- Saves memory and avoids unnecessary computation

3 Actions

- Operations like `count()`, `collect()`, `saveAsTextFile()`
- **Trigger actual execution**
- Spark runs all transformations together

Example

```
scala
```

```
words.count()
```

4 DAG Execution

- Spark divides work into **stages**
- Stages are split into **tasks**
- Tasks run in parallel on worker nodes

A **DataFrame** is:

- A **table-like structure** (rows & columns)
- Similar to a **database table**
- Built on top of RDDs
- Optimized by Spark automatically

2. DataFrames

A DataFrame is a distributed collection of data organized into named columns, similar to a relational database table.

Student Data:

id	name	marks
1	Ram	80
2	Sita	90
3	Hari	75

Advantages

- ✓ Less code
- ✓ SQL-like operations
- ✓ Faster than RDD
- ✓ Beginner-friendly

Output:

```
df = spark.read.csv("students.csv", header=True)

df.select("name", "marks").show()
```

nginx

```
Ram    80
Sita    90
Hari    75
```

Datasets

RDDs

- Functional Programming
- Type-safe

Dataframes

- Relational
- Catalyst query optimization
- Tungsten direct/packed RAM
- JIT code generation
- Sorting/suffling without deserializing

Encoder

Spark

- Combination of RDD + DataFrame
- Structured data with **compile-time type safety**

RDDs vs DataFrames vs Datasets in PySpark: What's the Difference?

RDDs

```
lang_rdd =  
  sc.parallelize  
    (("Python", 95),  
     ("R", 65),  
     ("Julia", 40))  
popular_langs =  
  lang_rdd.filter  
    lambda x: x(1) >= 60  
print popular_langs  
collect()
```

DataFrames

```
df = spark  
createDataFrame(  
  (("Python", 95),  
   ("R", 65),  
   ("Julia", 40)),  
  ['Language', 'Popularity'])  
df.filter  
  (df.Popularity > 60)  
show
```

Datasets

```
case class  
  Language(name: String,  
           popularity: Int)  
val ds =  
  Seq(Language("Python",  
               95), Language("R",  
                             65)).toDS()  
ds.filter(_popularity  
  > 60).show
```

🔑 Major Differences (Quick Table)

Feature	RDD	DataFrame	Dataset
Schema	❌ No	✅ Yes	✅ Yes
Type Safety	✅ Yes	❌ No	✅ Yes
Optimization	❌ No	✅ Yes	✅ Yes
Ease of Use	❌ Hard	✅ Easy	⚠️ Medium
Performance	⚠️ Slow	✅ Fast	✅ Fast
SQL Support	❌ No	✅ Yes	✅ Yes

Type safety means **errors related to data types are caught at compile time, not at runtime.**

👉 Example:

If a variable is defined as `Int`, you **cannot** accidentally assign a `String` to it.

3. Spark SQL

Spark SQL allows you to:

- Query data using **SQL commands**
- Work with DataFrames as SQL tables

Spark SQL is a Spark module that allows querying structured data using SQL queries.

Register DataFrame as a table:

```
python  
  
df.createOrReplaceTempView("students")
```

SQL Query

```
sql  
  
SELECT name, marks  
FROM students  
WHERE marks > 80;
```

Output:

```
nginx  
  
Sita 90
```

Key Advantage

👉 People who know SQL can use Spark without learning programming

Relationship Between RDD, DataFrame & Spark SQL

Spark SQL



DataFrame



RDD

- **RDD** → low-level (more control)
- **DataFrame** → structured & optimized
- **Spark SQL** → easiest (SQL based)

When to Use What?

Scenario	Use
Low-level control	RDD
Structured data	DataFrame
SQL queries	Spark SQL
Machine Learning	DataFrame
Big batch jobs	Spark
Simple analytics	Spark SQL

Apache Spark is faster than Hadoop MapReduce because it performs in-memory computation and provides easy-to-use abstractions like RDDs, DataFrames, and Spark SQL.

Case Study: University Exam Result Analysis using Apache Spark

A University (like *PU*) conducts exams for thousands of students every semester.

The exam section wants to quickly analyze results such as:

- Pass / Fail count
- Subject-wise average marks
- List of toppers
- Overall performance trend

Because the data size is very large, traditional tools (Excel, normal DB) become slow.

👉 **Apache Spark is used to solve this problem.**

Data Available

A CSV file stored in **HDFS**:

student_id	name	subject	marks
101	Ram	DBMS	78
102	Sita	DBMS	92
103	Hari	DBMS	35
101	Ram	Java	85
102	Sita	Java	88

Why Spark?

- Data size is **huge**
- Fast result required
- Repeated analysis (averages, filtering, ranking)
- Spark processes data **in-memory** → very fast

Step 1: Load Data (DataFrame)

python

```
df = spark.read.csv("hdfs://results.csv", header=True)
```

Now data is in **DataFrame form** (table-like).

Step 2: Pass / Fail Analysis (DataFrame)

python

```
pass_students = df.filter(df.marks >= 40)  
fail_students = df.filter(df.marks < 40)
```

✓ Quickly separates pass and fail students.

Step 3: Subject-wise Average Marks (Spark SQL)

python

```
df.createOrReplaceTempView("results")
```

sql

```
SELECT subject, AVG(marks) AS avg_marks  
FROM results  
GROUP BY subject;
```

✓ Helps teachers identify **difficult subjects**.

Step 5: Repeated Queries (RDD In-Memory Advantage)

Since data stays in **memory**, repeated queries like:

- ranking
- comparisons
- statistics

run **much faster** than Hadoop MapReduce.

Step 4: Find Toppers (Spark SQL)

sql

```
SELECT name, subject, marks  
FROM results  
ORDER BY marks DESC  
LIMIT 3;
```

✓ Used for merit list and awards.

Final Outcome

Using Apache Spark, the college:

- **Generates results within minutes**
- **Avoids disk-based delays**
- **Performs advanced analysis easily**
- **Publishes reports quickly**

Time-Series Databases and its implementation

A Time Series Database is a database specially designed to store, manage, and analyze data points that are indexed by time. Each record contains:

- *Timestamp (time)*
- *One or more measured values*

A Time Series Database stores data in which each value is associated with a timestamp and is optimized for time-based queries.

Key Characteristics

- Data is **ordered by time**
- High-volume **write operations**
- Efficient **range queries** (last hour, last day, last month)
- Optimized for **aggregation over time**
- Mostly **append-only** (new data added continuously)

TIME SERIES DATABASE



cassandra



druid



InfluxDB



elasticsearch

Real-Life Examples

1 Weather Monitoring System

A weather station records temperature every minute.

time	temperature
10:00	25°C
10:01	25.2°C
10:02	25.3°C

This data is **time-based** → stored in a **Time Series Database**.

2 Server Monitoring (Very Common)

Companies monitor server CPU usage every second.

time	CPU_Usage
12:00:01	45%
12:00:02	48%
12:00:03	55%

Used to:

- Detect performance issues
- Predict failures

Why Not Normal Databases?

Traditional databases:

- Are slower for time-range queries
- Not optimized for high-frequency data
- Use more storage

Time Series DBs:

- ✓ Faster
- ✓ Less storage
- ✓ Better compression

Popular Time Series Databases (Examples)

Database	Used For
InfluxDB	IoT, monitoring
Prometheus	System & container monitoring
TimescaleDB	Time-series on PostgreSQL
OpenTSDB	Hadoop-based TSDB
Amazon Timestream	Cloud time-series data

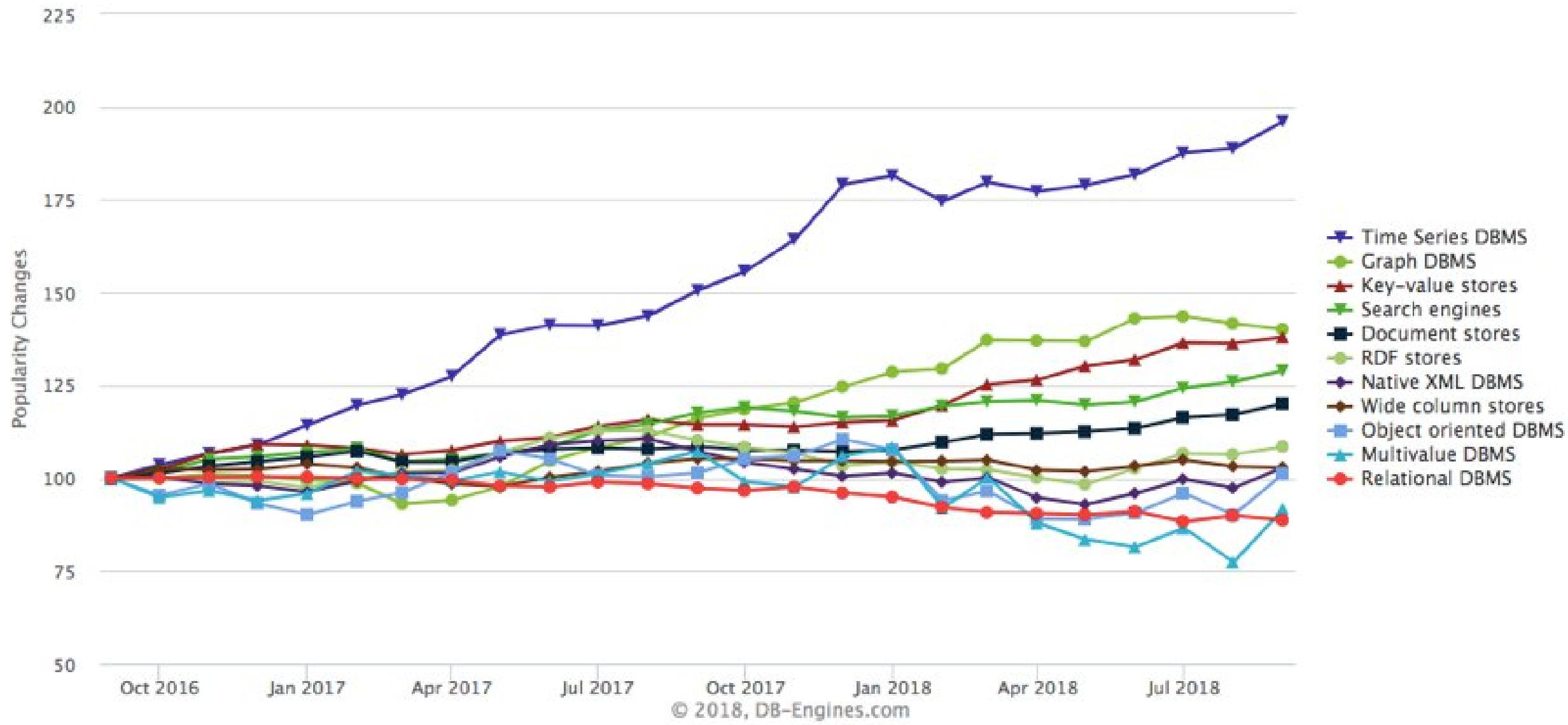
Question: Average temperature in last 1 hour

sql

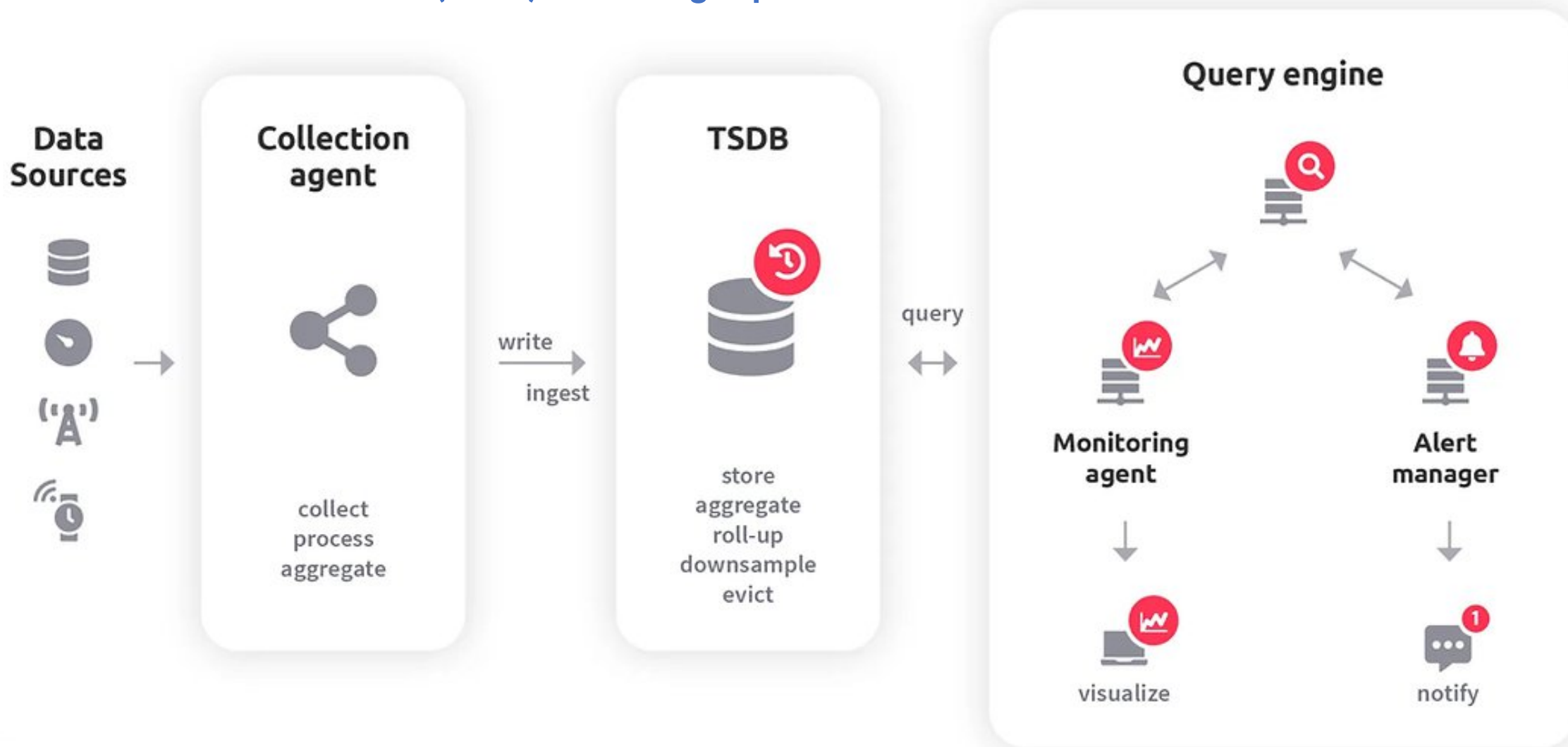
```
SELECT AVG(temperature)
FROM weather
WHERE time > now() - 1 hour;
```

Time Series Databases are used to store and analyze time-dependent data such as sensor readings, stock prices, and server metrics.

Trend of the last 24 months



Time Series Database (TSDB) – Working Explanation



↻ Overall Flow

Data is collected → stored in TSDB → queried → visualized or alerts are generated.

A Time Series Database is designed to store and analyze time-stamped data such as CPU usage, temperature, network traffic, or sensor readings.

1 Data Sources

- Devices, servers, applications, sensors, IoT, logs
- Continuously generate **time-based metrics**

✚ *Example: CPU usage every second*

2 Collection Agent

- Collects data from sources
- Performs basic processing:
 - Filtering
 - Aggregation
 - Formatting

✚ *Example: Telegraf, Prometheus agent*

6 Alert Manager

- Continuously checks rules
- Triggers alerts when thresholds are crossed

✚ *Example: CPU usage > 90% → Alert*

3 TSDB (Time Series Database)

- Stores time-stamped data efficiently
- Key operations:
 - **Store** incoming data
 - **Aggregate** (hourly/daily averages)
 - **Roll-up** old data
 - **Downsample** (reduce data size)
 - **Evict** outdated data

✚ *Examples: InfluxDB, Prometheus, TimescaleDB*

4 Query Engine

- Executes queries on stored time-series data
- Supports:
 - Fast time-range queries
 - Aggregations

5 Monitoring Agent

- Uses query engine to fetch metrics
- Displays data in charts and dashboards

✚ *Example: Grafana*

What is InfluxDB?

InfluxDB is an **open-source Time Series Database** designed specifically to **store and analyze time-stamped data** efficiently.

It is widely used for:

- Server & network monitoring
- IoT sensor data
- Application performance monitoring
- Smart meters, weather data

How InfluxDB Handles Time Series Data

InfluxDB stores data in a **special time-series format**, not like normal tables.

Each data point contains:

1. **Timestamp** (mandatory)
2. **Measurement** (similar to table name)
3. **Tags** (indexed metadata)
4. **Fields** (actual values)



InfluxDB monitoring InfluxDB

Avg System Load



Queries Executed

397

Writes Executed

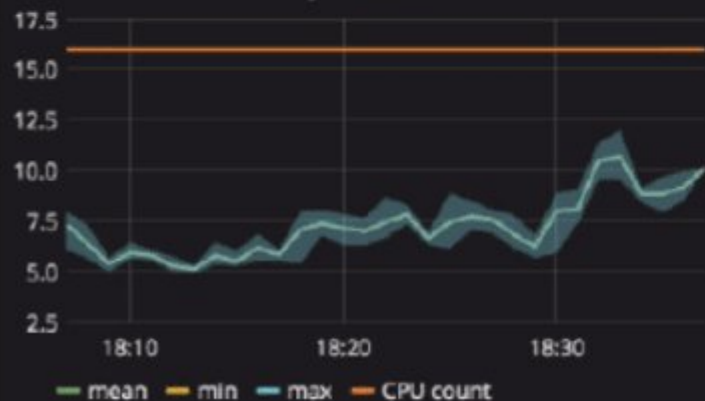
57716

Series Count

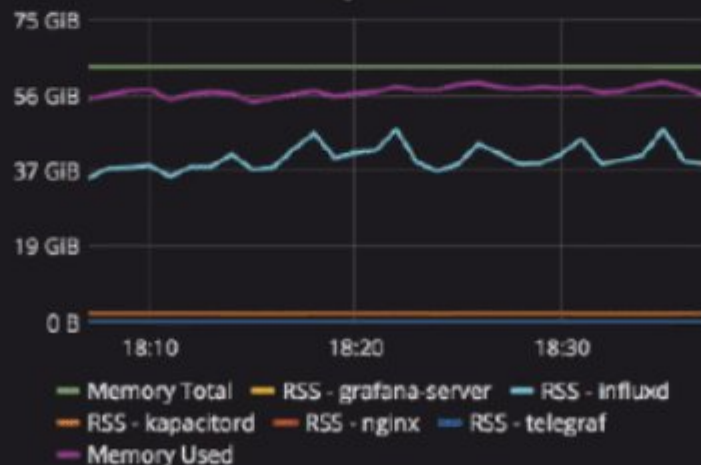
370950

System

System Load



Memory Utilization



CPU



InfluxDB Data Model

1 Measurement

- Logical container of time series data
- Similar to a table

Example:

```
nginx  
  
temperature
```

2 Timestamp

- Exact time when data is recorded
- Stored with high precision (ns, ms, s)

Example:

```
yaml  
  
2025-01-10 10:05:00
```

3 Tags (Indexed → Fast Search)

- Used for filtering
- Stored as **strings**
- Indexed automatically

Examples:

```
ini  
  
location = "Kathmandu"  
device = "sensor1"
```

4 Fields (Actual Values)

- Stored as values (int, float, boolean)
- Not indexed

Examples:

```
ini  
  
value = 25.6  
humidity = 60
```

Example: Weather Monitoring System

Data Collected Every Minute

Measurement: weather

time	location (tag)	temperature (field)	humidity (field)
10:00	KTM	25.2	60
10:01	KTM	25.4	61
10:02	KTM	25.6	62

Writing Data into InfluxDB

text

```
weather,location=KTM temperature=25.2,humidity=60 1736503200
```

Format:

sql

```
measurement,tag_key=tag_value field_key=field_value timestamp
```

Querying Time Series Data (InfluxQL)

Example 1: Last 5 minutes temperature

sql

```
SELECT temperature
FROM weather
WHERE time > now() - 5m;
```

Example 2: Average Temperature (Last 1 Hour)

sql

```
SELECT MEAN(temperature)
FROM weather
WHERE time > now() - 1h;
```

Example 3: Group by Time (Downsampling)

sql

```
SELECT MEAN(temperature)
FROM weather
GROUP BY time(10m);
```

✓ Reduces large data into summaries

How InfluxDB Is Optimized for Time Series

1 Time-based Storage

- Data is stored **sorted by time**
- Fast range queries

4 Continuous Queries

- Automatically creates summarized data

Example:

```
sql
```

```
Hourly average from minute data
```

3 Retention Policy

- Automatically deletes old data

Example:

```
kotlin
```

```
Keep data only for 30 days
```

2 Compression

- Similar timestamps stored efficiently
- Uses **Time-Structured Merge Tree (TSM)**

Real-World Use Case

Scenario: Monitor 10,000 IoT devices sending data every 5 seconds.

Solution:

1. **Database:** InfluxDB (handles high write throughput).
2. **Schema:**

plaintext

```
measurement: device_metrics
tags: device_id, location
fields: temperature, humidity
timestamp: auto-generated
```

3. **Query:** Alert if any device exceeds 30°C:

sql

```
SELECT * FROM device_metrics
WHERE temperature > 30 AND time > now() - 5m
```

Summary

- TSDBs excel at **time-ordered data** with fast writes/reads.
- **Key Tools:** InfluxDB (simple), TimescaleDB (SQL-powered), Prometheus (metrics).
- **Implementation Steps:**
 1. Define schema (tags/fields).
 2. Ingest data (often via HTTP/APIs).
 3. Query with time-range filters.

Server Monitoring

- CPU usage collected every second
- Stored in InfluxDB
- Visualized using **Grafana**

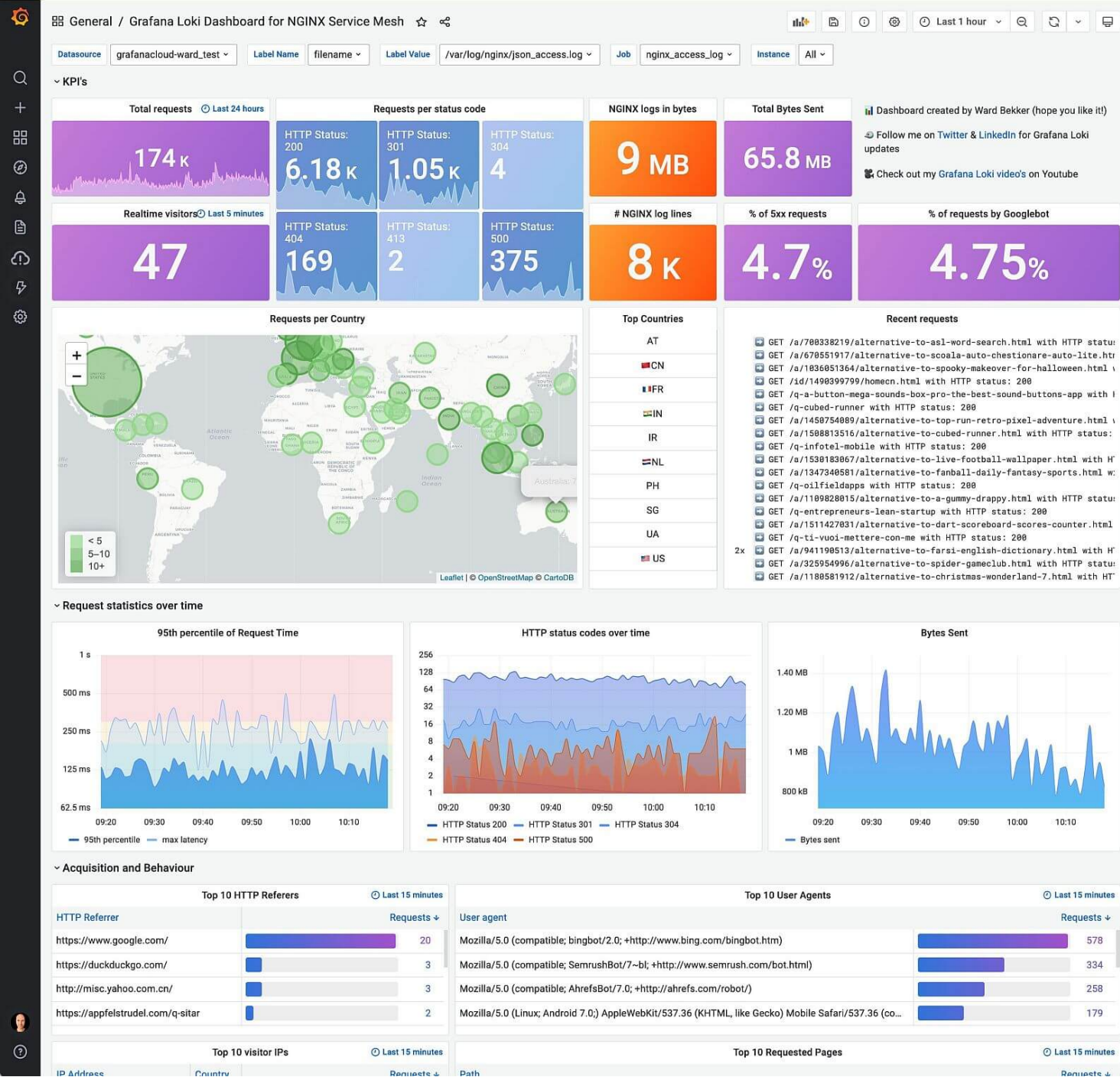
Real-Life Use Case

Example query:

sql

```
SELECT MAX(cpu_usage)
FROM cpu
WHERE time > now() - 10m;
```

Traditional DB	InfluxDB
Row-based storage	Time-based storage
Slower time queries	Optimized time queries
Manual cleanup	Retention policies



vSphere Overview

Datastore Summary 10

CL-LON Overview

RAM Usage
16.93%

Datastores - Usage Capacity

The graph shows a constant usage level of approximately 70-75% throughout the day from 09:00 to 14:00. The y-axis is labeled 'Usage Average %' and ranges from 0 to 100. The x-axis shows time in hourly increments.

Time	Usage Average %
09:00	~72
10:00	~72
11:00	~72
12:00	~72
13:00	~72
14:00	~72

Cluster Storage Adapter

ms

50 ms

0 ms

09:00 10:00 11:00 12:00 13:00 14:00

ZEXT-NET... ZEXT-QNA... ZINT-ESXi... ZINT-ESXi... ZINT-ESXi2... ZINT-ESXi2... ZINT-ESXi3... ZINT-ESXi3... ZINT-ESXi3... vsanDatast...

▼ Hypervisor Status

Hypervisor Memory

Hypervisor Net Usage

vCenter CPU/RAM

Memory Usage Avg. %

Time	Java	Other	OS	Free
09:00	45%	45%	5%	5%
10:00	45%	45%	5%	5%
11:00	45%	45%	5%	5%
12:00	45%	45%	5%	5%
13:00	45%	45%	5%	5%
14:00	45%	45%	5%	5%

	09:00	10:00	11:00	12:00	13:00	14:00
						Last Min
esxi-zion-001.jorgedelacruz.es mem usage average						68.9% 68.6%
esxi-zion-002.jorgedelacruz.es mem usage average						47.4% 47.3%

Time (min)	CPU Usage (%)
0	0
1	20
2	40
3	60
4	80
5	100
6	80
7	100
8	80
9	100
10	80

The graph, titled 'vCenter CPU/RAM', displays usage percentages on the y-axis (0% to 100%) against time on the x-axis (09:00 to 14:00). The CPU usage is represented by a blue line, and the RAM usage is represented by a green line. Both lines remain consistently low, generally staying below 10% throughout the entire period.

Virtual Machine CPU Usage in %

Virtual Machine Memory Usage Avg.

Virtual Machine CPU READY ms

Virtual Machine Network Usage Avg (Kbps)

Memory Usage Avg. %

09:00 10:00 11:00 12:00 13:00 14:00

AD-DNS-001 AD-DNS-002 AFA-NimOS-5.2.1.0
HPE-STOREONCE-VSA-G4 HPE-STOREONCE-VSA-G4-002
MINIO-SERVER NGINX-001 NGINX-002 NGINX-004

A line graph titled 'Net Usage Avg.' showing network usage in MB/s over time. The y-axis ranges from 0 kB/s to 4 MB/s. The x-axis shows time from 09:00 to 14:00. A sharp spike in usage is visible at 12:00, reaching approximately 4 MB/s. Red arrows point to the baseline usage level at 09:00, 10:00, 11:00, 13:00, and 14:00.

AD-DNS-001 AD-DNS-002 AFA-NimOS-5.2.1.0
HPE-STOREONCE-VSA-G4 HPE-STOREONCE-VSA-G4-002
MINIO-SERVER NGINX-001 NGINX-002 NGINX-004

Net Usage Avg.

4 MB/s

2 MB/s

0 KB/s

09:00 10:00 11:00 12:00 13:00 14:00

AD-DNS-001 AD-DNS-002 AFA-NimOS-5.2.1.0

HPE-STORAGE-VSA-G4 HPE-STORAGE-VSA-G4-002

NGINX-SERVER NGINX-001 NGINX-002 NGINX-004

Virtual Machine Disk Performance (Read VMFS Block IOPS)

Virtual Machine Disk Performance (Write VMFS Block IOPS)

Virtual Machine Disk Performance (Block Latency Total)

[illegible]

Virtual Machine Disk Performance (Block Latency Total)

Blockchain Databases (Decentralized Storage)

A blockchain database is a distributed, append-only database where data is stored in blocks, each block is cryptographically linked to the previous one, and shared across many nodes instead of being controlled by a single authority.

A blockchain database is a decentralized database where records are stored in blocks linked using cryptography, making the data immutable and tamper-resistant.

Traditional databases work well when there is trust in a central owner (bank, company, server). Blockchain is needed when trust is low or distributed.

Problems with Traditional Databases

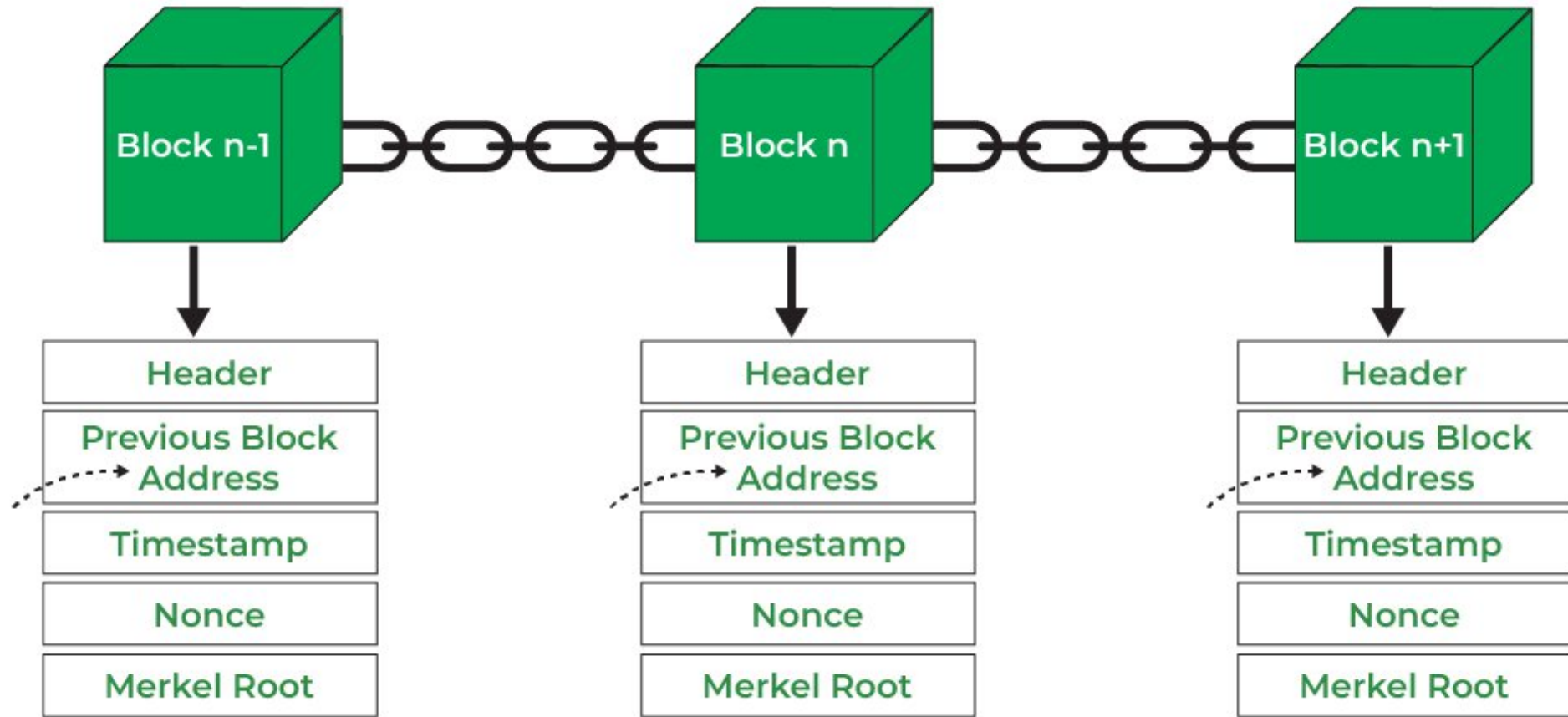
- Single owner controls data
- Data can be modified or deleted
- Single point of failure
- Difficult to audit changes

Blockchain Solves This By

-  No single owner (decentralized)
-  Data cannot be changed once added
-  All changes are transparent
-  Very hard to hack or manipulate

Example: Land Ownership Records

- If land records are in a **central database**, corruption or illegal modification is possible
- With **blockchain**, once ownership is recorded, it **cannot be altered secretly**
- Everyone can verify the history of ownership



- Each block contains transaction data, timestamp, previous block hash, and current hash
- The previous hash links blocks together
- If data changes, hash changes → tampering detected

Key idea:

🔗 Blocks are securely linked using cryptographic hashes

- **Block Header:** Stores block metadata
- **Previous Hash:** Links blocks securely
- **Timestamp:** Records block creation time
- **Nonce:** Used to generate valid hash
- **Merkle Root:** Represents all transactions

1. Block Header

- **Meaning:** The main section of a block that stores important metadata.
- **Purpose:**
 - Used to uniquely identify the block.
 - Contains information required for block validation and mining.
- **Includes:** Timestamp, Nonce, Merkle Root, and Previous Hash.

2. Previous Hash Address

- **Meaning:** The hash value of the previous block.
- **Purpose:**
 - Links blocks together to form a **chain**.
 - Ensures **immutability**—if a previous block is altered, all following blocks become invalid.
- **Example:**
 - Previous Hash = 000000ab34f...

3. Timestamp

- **Meaning:** Records the exact time when the block is created.
- **Purpose:**
 - Maintains chronological order of blocks.
 - Helps in tracking and verification.
- **Example:**
 - 2026-01-20 10:45:30

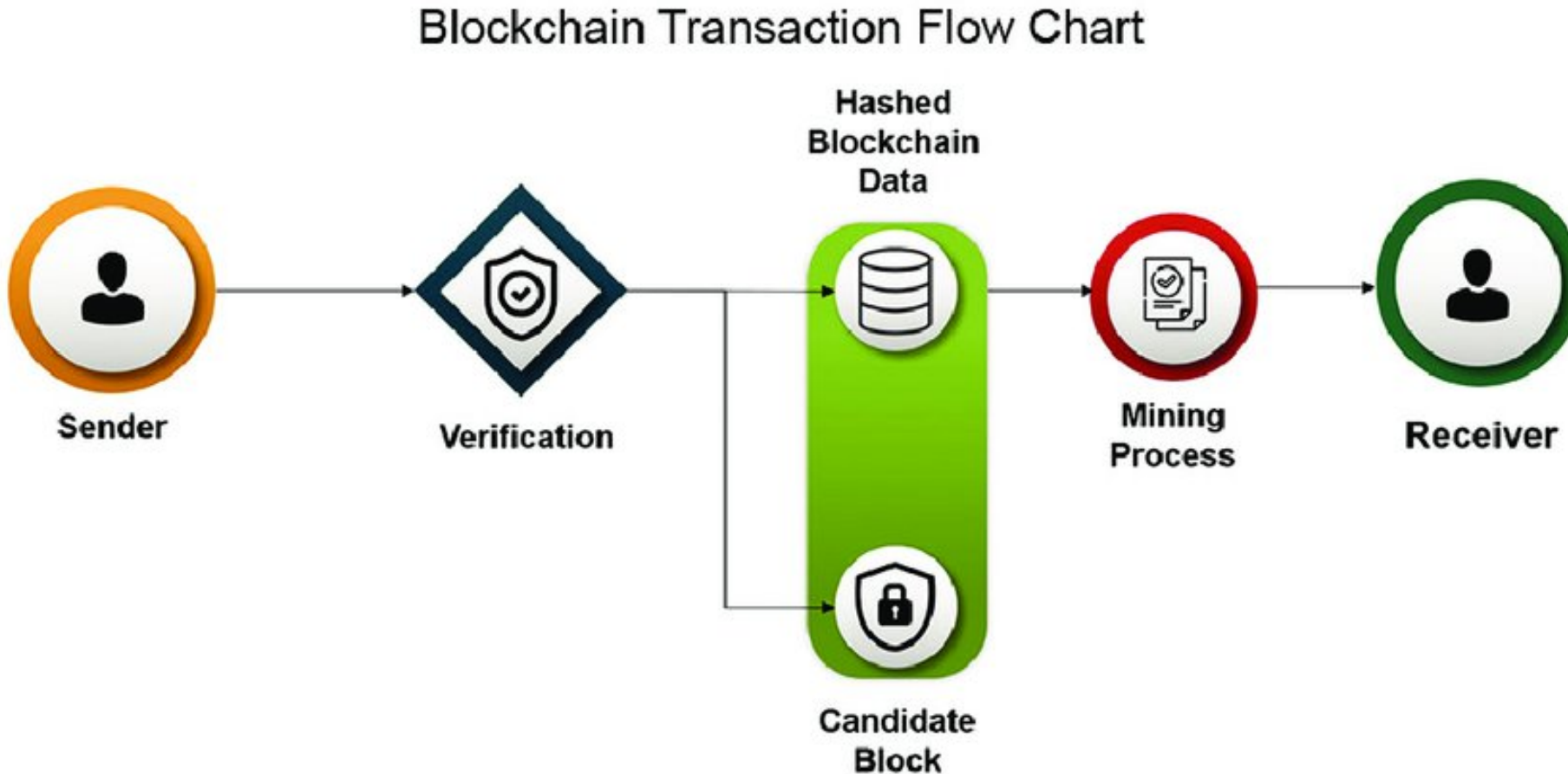
4. Nonce (Number used once)

- **Meaning:** A random number used during mining.
- **Purpose:**
 - Helps miners find a valid hash under Proof of Work.
 - Changed repeatedly until difficulty requirements are met.
- **Example:**
 - Nonce = 845732

5. Merkle Root

- **Meaning:** A single hash representing all transactions in the block.
- **Purpose:**
 - Ensures integrity and security of transactions.
 - Any change in transaction data changes the Merkle Root.
- **Example:**
 - Merkle Root = Hash(Hash(T1+T2) + Hash(T3+T4))

How a transaction becomes part of the blockchain



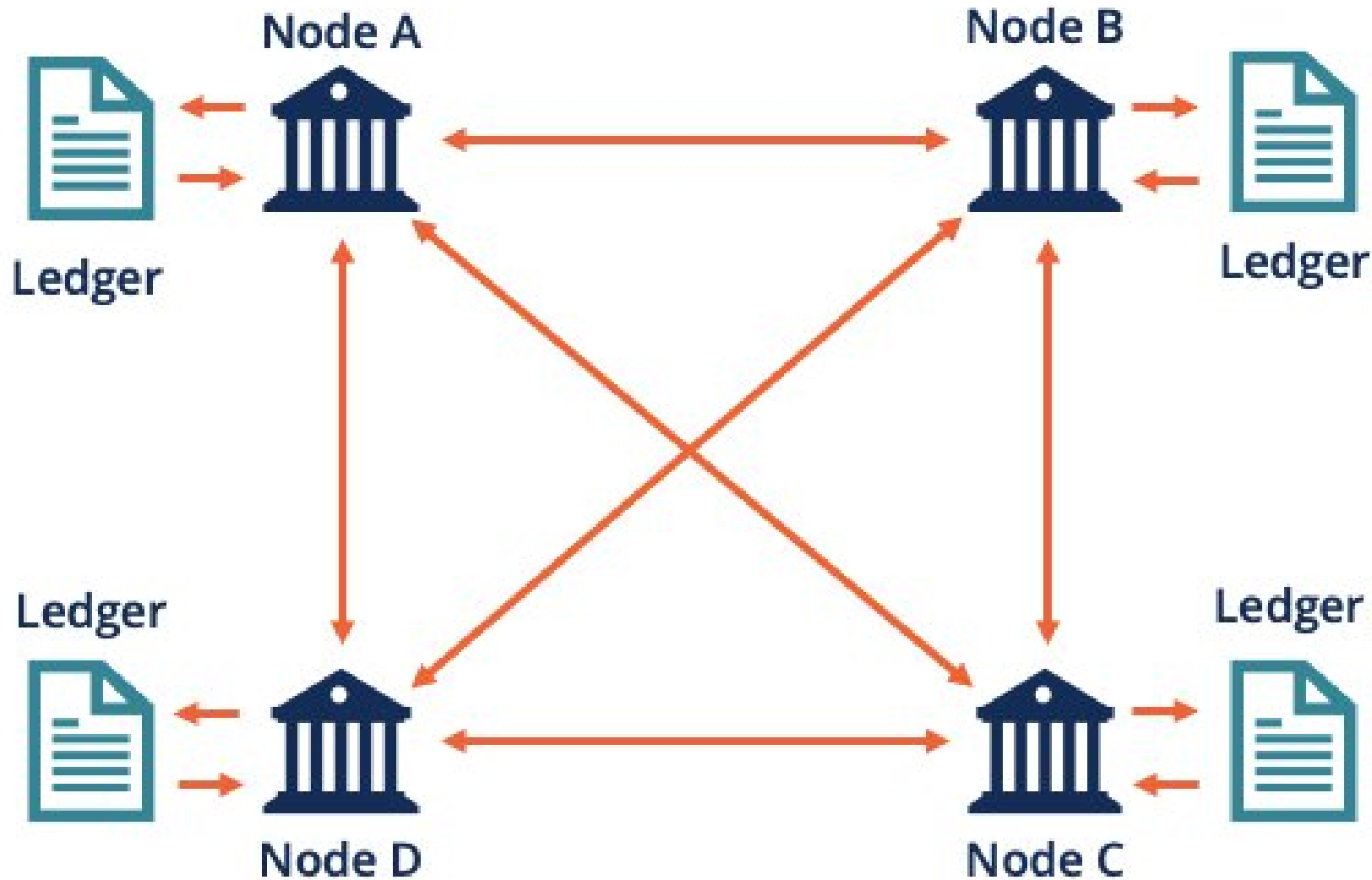
1. **User creates a transaction**
2. **Network nodes verify it**
3. **Verified transactions form a block**
4. **Block is added to the blockchain**

Key idea:

➡ Only verified transactions are permanently stored

Distributed Ledgers

Blockchain copies stored on multiple nodes



- Every node has a complete copy of the blockchain
- No central server
- If one node fails, others continue

Key idea:

🔗 Blockchain is decentralized and fault-tolerant

Where Blockchain Databases Are Used

- 💰 Cryptocurrency (Bitcoin, Ethereum)
- 📦 Supply Chain Tracking
- 🏥 Medical Records
- 🗳️ Voting Systems
- 🏠 Land Registry
- 📄 Smart Contracts

Popular Blockchain Databases / Platforms

- Ethereum
- Hyperledger Fabric
- Corda
- BigchainDB (Blockchain + Database features)

Advantages

- High security
- Data integrity
- Transparency
- No single point of failure

Limitations

- Slower than traditional DBs
- High storage usage
- Complex implementation

1. Data is Stored in Blocks

Each block contains:

- **Data** (e.g., transaction records, documents)
- **Hash** (a unique fingerprint of the block)
- **Previous block's hash** (links blocks like a chain)

How Blockchain Databases Work

2. Decentralized Storage (Peer-to-Peer Network)

- Instead of one server, data is stored across **many nodes (computers)**.
- Each node keeps a copy of the blockchain.

3. Consensus Mechanism (Agreement Rule)

Before adding a new block, nodes must **agree** (e.g., via **Proof of Work** or **Proof of Stake**).

4. Immutability (Tamper-Proof)

- Changing old data requires changing **all future blocks** (nearly impossible).
- Makes fraud very difficult.

```
{
  "block_id": "0003",
  "data": {
    "student": "Alice",
    "degree": "Computer Science",
    "year": "2023",
    "issuer": "XYZ University"
  },
  "hash": "a1b2c3...",
  "prev_hash": "x9y8z7..." // Links to the previous block
}
```

Step 1: Land Sale Transaction

Ram sells land (Plot No. 25) to Sita.

Transaction data:

yaml

```
Seller: Ram
Buyer: Sita
Plot No: 25
Date: 2025-01-10
```

Step 2: Transaction Broadcast

- Transaction is broadcast to all blockchain nodes
- Nodes check:
 - Is Ram the real owner?
 - Is the land already sold?

Step 3: Validation (Consensus)

- Majority of nodes approve the transaction
 - Uses a **consensus algorithm**
- ✓ Prevents fake or duplicate ownership

Step 4: Block Creation

- Approved transaction is added to a new block
- Block includes:
 - Transaction data
 - Previous block hash
 - Timestamp

Step 5: Block Added to Blockchain

- Block is linked to previous block
- Once added, record becomes **permanent**

Step 6: Distributed Storage

- Updated blockchain copy stored on **all nodes**
- Anyone can verify ownership history

Transaction → Verification → Block → Chain → Distributed Storage

Homework

12. Explain the working principles of the MapReduce Programming Model. Describe the roles of Map and Reduce functions in distributed data processing, and illustrate your explanations with a real world use case. [PU, 2025]
13. Explain the working mechanism of blockchain technology with appropriate illustrations. Why is a blockchain based database considered more secure than traditional centralized database systems? [PU, 2025]
14. Explain the architecture of HDFS. How is read and write performed in HDFS? [SOE, Pre-board]